Adaptive pricing in imperfect competitive markets: a reinforcement learning approach

Chiara Pizzetti, Karina Trelles
University of Strasbourg

Introduction

In this work, we replicate two imperfect competition models to analyze the pricing dynamics in a market with differentiated goods, such as coffee capsules. We first consider a duopoly in which two sellers update their prices based on their own past prices and the prices observed from their competitors. This approach enables us to capture the mechanisms of gradual adjustment, strategic reactions, and potential convergence dynamics, cycles, or instability, depending on the interaction parameters. Afterwards, we extend the analysis to a discrete oligopoly with three agents and similar update rules, intending to assess how the introduction of an additional competitor alters the trajectory of prices and the intensity of competition. The comparison between both models allows us to discuss how market structure influences competitive dynamics, system stability, and the emergence of complex behaviors in markets with strategic interactions among multiple agents.

From a theoretical perspective, the model is grounded in the canonical frameworks of imperfect competition, particularly in Bertrand competition with product differentiation and in the horizontal differentiation structure introduced by Hotelling. The price adjustment mechanism follows the logic of adaptive expectations and bounded rationality, whereby firms adjust their decision through partial-response rules rather than full optimization. This approach is supported by the literature on duopoly dynamics, which shows that simple reactive rules may generate convergence to Nash equilibrium, limit cycles, or even chaotic attractors, depending on the local stability conditions (Agiza & Elsadany, 2003; Bischi & Lamantia, 2022). The framework also aligns with recent contributions on algorithmic collusion and learning-based pricing strategies, in which reinforcement-learning agents interact strategically in repeated pricing environments (Calvano et al., 2020).

The models, therefore, replicate key characteristics of imperfect competition as described in Bertrand and Cournot frameworks, highlighting how firms strategically adjust prices in response to competitors' actions. Firms are assumed to adjust prices according to adaptive expectations, a concept rooted in bounded rationality and dynamic game theory, which allows capturing gradual learning and strategic behavior in a repeated interaction setting.

It is essential to note that, for implementation purposes, the model is deliberately simplified, as representing all the assumptions and complexities of the real market is extremely challenging. Nevertheless, to maintain a reasonable degree of realism, we use parameter values similar to those observed in the actual market, both for the average prices of coffee capsules and for the parameters related to price sensitivity and competitive interaction. These choices allow us to preserve the fundamental logic of strategic behavior among firms, even within a stylized modeling framework. In this sense, the aim is not to accurately predict the behavior of real firms, but to analyze the qualitative dynamics that emerge at different levels of competitive interaction.

Having established the theoretical framework and the scope of the model, we now describe the agents and the environment in which they interact. Each agent represents a seller who makes a discrete pricing decision. We consider two settings: the first simulates competition between two coffee capsule sellers, and the second extends the framework to three competing agents. The general structure of the environment is identical in both cases; the only difference across models lies in the definition of the state space, which expands with the number of agents. The components of the environment are described below:

1. *State* s_t: At each time step, an agent observes its own previous price and the competitor's previous price:

$$s_t{}^1 = (own_price_{t-1}, competitor_price_{t-1}) \quad (1)$$
$$s_t{}^2 = (own_price_{t-1}, competitor1_price_{t-1}, competitor2_price_{t-1}) \quad (2)$$

2. *Action* a_t*:* The agent selects a price from a predefined discrete set:

$$a_t \in \{p_1, p_2, \ldots, p_n\} \quad (3)$$

3. *Reward* r_t: The reward corresponds to the profit obtained in the current period, calculated as:

$$r_t = (own_price - cost) \cdot demand \quad (4)$$

4. *Demand Dynamics:* The demand is modeled as a linear function of both the agent's own price and the competitor's price:

$$Demand = a + b \cdot own_price + c \cdot competitor_price \quad (5)$$

The detailed functioning of the environment, including the agents' interactions and simulation steps, is described in the methodology section. Here, we focus on the demand function in the duopoly model. Following the principle of parsimony, we adopt a linear

[1] Duopoly
[2] Discrete oligopoly

specification to characterize how market demand responds to each agent's pricing decisions. The demand for agent *i* is modeled as:

$$D_i = a - bp_i + c(p_j - p_i) \quad (6)$$

Where:

- D_i is the quantity demanded from agent (*i*),
- p_i is the price set by agent (*i*),
- p_j is the price set by its competitor,
- a represents the base market demand,
- b is the price sensitivity parameter of demand,
- c captures the effect of the competitor's price on the agent's demand.

By reorganizing the expression, we obtain an equivalent equation that facilitates its economic interpretation:

$$D_i = a - (b + c)p_i + cp_j \quad (7)$$

That equation captures two important effects:

1. *Sensitivity to own price:* the term $(-(b + c)p_i)$ indicates that demand decreases when the agent raises its own price. In this model, we use (b = 1.8) y (c = 0.5), so the effective sensitivity is 2.3.
2. *Sensitivity to competitor price*: The term $c\mathrm{p_j}$ shows that the agent's demand increases when the competitor raises its price.

This behavior captures substitution effects typical of price competition models, Bertrand-type duopoly, consumers shift toward the relatively cheaper alternative.

In general, this demand function captures, simply and linearly, the effects of price competition. The equation ensures that an increase in a firm's own price reduces its demand, while an increase in a competitor's price raises its demand. The magnitude of both effects is governed by parameters representing customer behavior in response to price changes. In summary, the parameters used in the demand function are a base market demand of 20 (*a*), a negative sensitivity to own price of 1.8 (*b*), and a competitor price effect of 0.5 (*c*).

Methodology

To study how autonomous firms may learn competitive pricing strategies, we implement and compare two reinforcement learning algorithms: Independent Q-Learning (IQL) and

Deep Q-Learning (DQN). We train two pricing agents and later three, using Q-learning and DQN. The explanations that follow are based on the two-agent environment. We will later outline the modifications required when the framework is extended to three agents.

The Q-learning hyperparameters are set as follows: the learning rate α controls how quickly the Q-values are updated; if α is too high, the agent reacts too quickly to short-term fluctuations in profit; if it is too low, learning becomes extremely slow. A value of 0.1 provides a good balance: the agent updates its Q-values gradually, allowing the pricing strategy to improve steadily.
The discount factor γ determines how much future rewards influence the agent's decisions. With γ = 0.95, the agent cares more about long-term profitability. This is crucial in competitive pricing: a high price may give a good immediate payoff, but it can also encourage the competitor to undercut in the next step. A high γ encourages more strategic behavior and promotes forward-looking.
In this setting, the ε-greedy strategy controls the exploration-exploitation trade-off. By setting the initial value of ε equal to 1.0, the agent begins training with full exploration. The minimum value equal to 0.05 ensures that a small amount of randomness is maintained throughout the learning process, preventing the agent from becoming fully predictable and exploitable. Finally, an ε decay of 0.9995 allows exploration to decrease gradually throughout the episodes.
In this setup, the training process runs for 10 000 episodes, and for each episode, a limit of 50 steps is imposed.

Each agent maintains its own Q-table and, at each step of the interaction, updates it as follows:

$$Q(s,a) \leftarrow Q(s,a) + \alpha(r + \gamma \max_{a'} Q(s',a') - Q(s,a)) \quad (8)$$

Each agent learns independently, considering:

- The agent's own previous price.
- The competitor's previous price.

Given that, the state representation for the 2-agent is defined in equation (1). The training loop implements IQL for two pricing agents interacting in a competitive environment. At the beginning of each episode, the environment is reset, and each agent receives its initial state, represented as a pair of prices: the agent's previous price and the competitor's previous price. This 2-dimensional state is mapped to a unique integer index so it can be used to access the corresponding row in the Q-table. During each time step, both agents select an action using an ε-greedy policy. With probability ε, the agent explores by choosing a random price; with probability 1 - ε, it exploits its current knowledge by selecting the action with the highest Q-value. Given the chosen actions, the environment returns the next state, and the individual returns for both agents. Each agent then updates its Q-table using the standard Q-learning update rule defined in equation (8). At the end of each episode, the cumulative rewards for both agents are stored to monitor learning

progress. Over the course of the training, the exploration rate ε decays gradually, ensuring that later, the agents increasingly exploit their learned knowledge to maximize rewards. Once training is completed, the greedy policy for each agent is extracted by selecting the action with the highest Q-value for every state in its Q-table. Finally, the learned policies are evaluated in a separate rollout to measure their average reward performance.

Later on, we extend the Q-learning approach by using Deep Q-Networks (DQN) to investigate whether a neural network approximation can improve training stability and efficiency. The hyperparameters set for the DQN implementation are the same as those defined in the Q-learning setting, but some differences arise.
First, the learning rate α is set to 0.01, which controls how quickly the neural network updates its weights after each gradient step.
Moreover, since DQN uses a neural network to approximate the Q-function, additional parameters have to be defined:

- Hidden size = 32: specifies the number of neurons in the hidden layer of the network.
- Buffer size = 5 000: means that the agent keeps up to 5 000 experiences, allowing it to sample past interactions and break correlation between consecutive samples.
- Batch size = 32: during training, the network is updated using mini-batches of 32 randomly selected experiences from the replay buffer. This stabilizes learning and provides smoother gradient updates.

Unlike Q-learning, DQN does not explicitly maintain Q-tables. Instead, the Q-values are approximated by the neural network.
Each agent receives a state input, which is the same as defined in equation (1).

The agents during the training process follow a standard independent DQN framework. At the beginning of each episode, the environment resets and each agent receives its initial state, which is fed into its neural network to estimate the Q-values for all possible actions. At each time step, agents choose actions using an ε-greedy policy: with probability ε, they explore by selecting a random action; otherwise, they exploit their current Q-network by choosing the action with the highest predicted value.
Once both agents choose actions, the environment returns the next state and the corresponding reward. Each agent stores the transition in its replay buffer:

$$(s_i, a_i, r_i, s'_i, d_i) \quad (9)$$

It is a memory structure that keeps track of past experiences, where:

- s_i: is the current state,
- a_i: is the action taken,
- r_i: is the reward received,
- s'_i: is the next state,
- d_i: indicates whether the episode has ended.

The replay buffer is needed for two main purposes:

- Breaking the correlation between consecutive experiences
- Reusing past experience

Once the buffer contains enough sample, for each sampled transition (equation before) the target Q-value is computed as:

$$y = r + \gamma \max_{a'} Q_\theta(s', a') \cdot (1 - d) \quad (10)$$

The network weights are updated via gradient descent to minimize the difference between the predicted and target values. After each episode, the exploration rate ε gradually decreases, shifting agents from exploration toward exploitation. Finally, the cumulative rewards obtained in each episode are recorded to monitor learning progress.

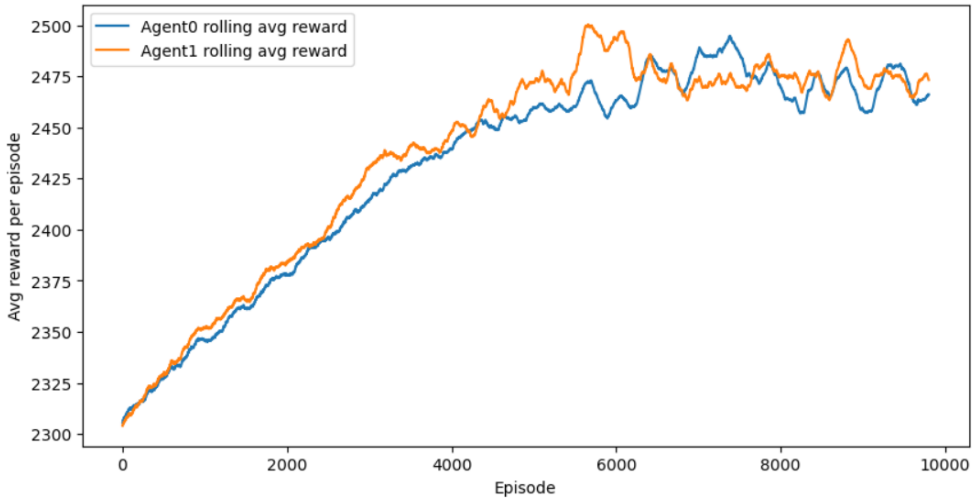
When extending Q-learning and DQN from two to three agents, the core methodology remains unchanged, but the complexity increases, as each agent now considers two competitors instead of one. The state is now represented in equation (2). In Q-learning, the state space and Q-tables grow larger, with three Q-tables to update instead of two. In DQN, each network's input must include information about two competitors, increasing state dimensionality and making the mapping to Q-values more complex.

Results

We now turn to the analysis of the simulation results, beginning with the duopoly model. This section presents how the two competing agents adjust their prices over time and how the market responds to these interactions. By examining the trajectories of prices, convergence patterns, and potential cyclical or unstable behaviors, we aim to highlight the dynamic properties that emerge from strategic interactions between the two firms. The findings provide a baseline for understanding the effects of competition in a simple two-agent setting before extending the analysis to the three-agent model.

Figure 1 reflects a well-structured convergence process under Q-learning within a symmetric competitive environment between two agents (Agent0 and Agent1). Both agents exhibit a persistent upward trend in their rolling average rewards, indicating a gradual refinement of action-value estimates and an increasingly accurate mapping of the state-action space. The initial high-volatility segment, driven by ε-greedy exploration and the stochasticity inherent in early Q-value updates, gives way to progressively smoother trajectories as learning stabilizes and exploitation becomes dominant.

Figure 1 Training Q-learning progress - duopoly

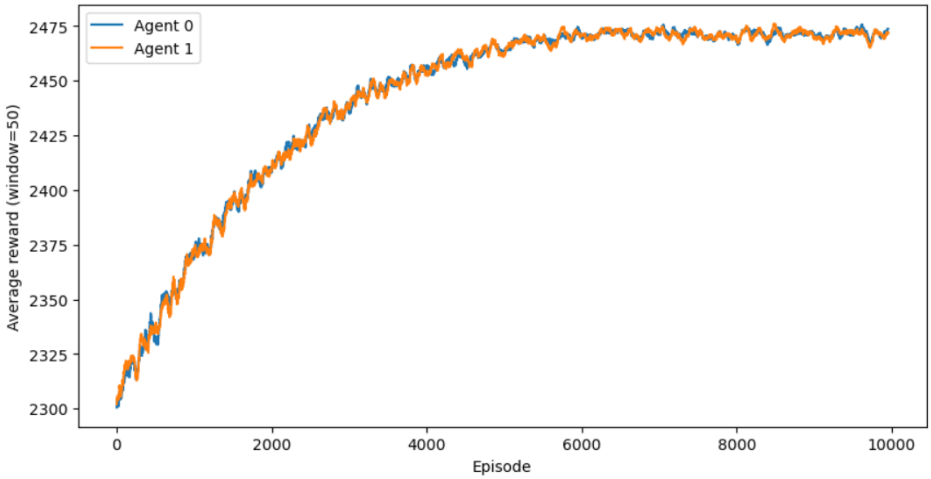


Despite the overall symmetry of the environment, a small and stable performance differential emerges between Agent0 and Agent1. From a technical perspective, such divergence can arise from asymmetric random initialization of Q-values, heterogeneity in pseudo-random action sampling, or uneven early exposure to high-reward states, all of which can induce path dependence in value-function estimates. However, the persistence yet narrowness of the gap suggests that both agents converge to closely aligned policies and that no structural advantage is embedded in the environment.

From an economic standpoint, the learning dynamics depicted in the figure mirror the adjustment processes found in repeated duopolistic price competition. The monotonic reward improvement reflects the agents' gradual movement toward pricing strategies that better internalize demand elasticities and competitive responses. As volatility declines and both reward trajectories stabilize near similar values, the system exhibits behavior characteristic of convergence to a stationary strategic equilibrium. This equilibrium may correspond to the Nash equilibrium of the underlying price game or, depending on the reward structure and information feedback, to supra-competitive price levels indicative of emergent tacit coordination.

Figure 2 displays the training dynamics of two reinforcement learning agents (Agente 0 and Agent 1) employing a Deep Q-Network (DQN) algorithm in a repeated competitive environment. The vertical axis reports the average reward computed over a rolling window of 50 episodes, while the horizontal axis represents the progression of training episodes.

Figure 2 Training DQN - duopoly

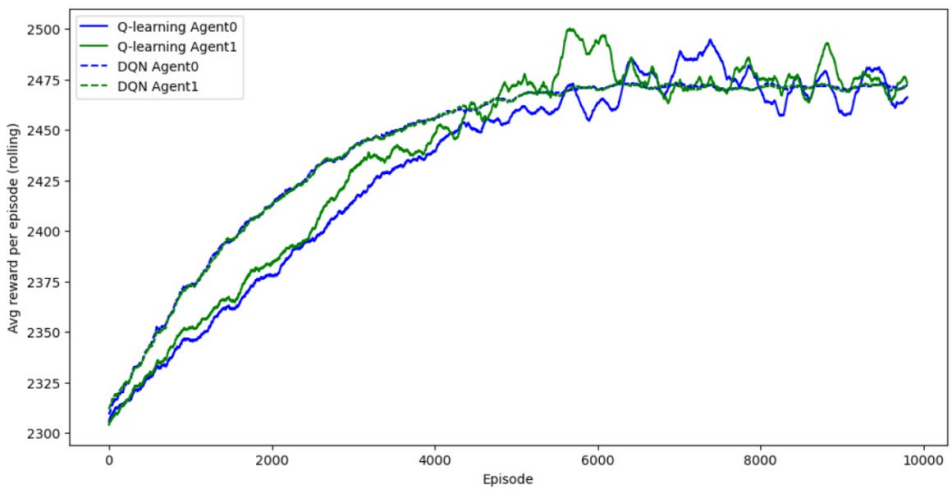


Both trajectories exhibit a monotonic upward trend during the initial stages of training, reflecting the agents' gradual improvement as they transition from exploratory behavior to more refined and profitable strategies. The curves begin to stabilize approximately between episodes 6 000 and 7 000, indicating convergence toward a near-stationary policy. The fact that both agents achieve almost indistinguishable learning paths and reward levels suggests that they operate under symmetric environmental and strategic conditions, resulting in comparable value-function updates and policy outcomes.

From an economic standpoint, the convergence of rewards indicates that both agents settle into a stable strategic pattern typical of a symmetric duopoly. In a price-competition setting, the upward reward trend reflects a gradual adjustment toward more profitable pricing strategies, while the stabilization around similar values for both agents suggests the emergence of mutually consistent behaviors. This outcome is compatible with convergence to a Nash equilibrium of the repeated game or, depending on the reward structure, to supracompetitive prices that reflect tacit coordination.

Figure 3 compare the learning speed, stability, and final performance of both methods.

Figure 3 Comparison training: Q-learning vs DQN - duopoly

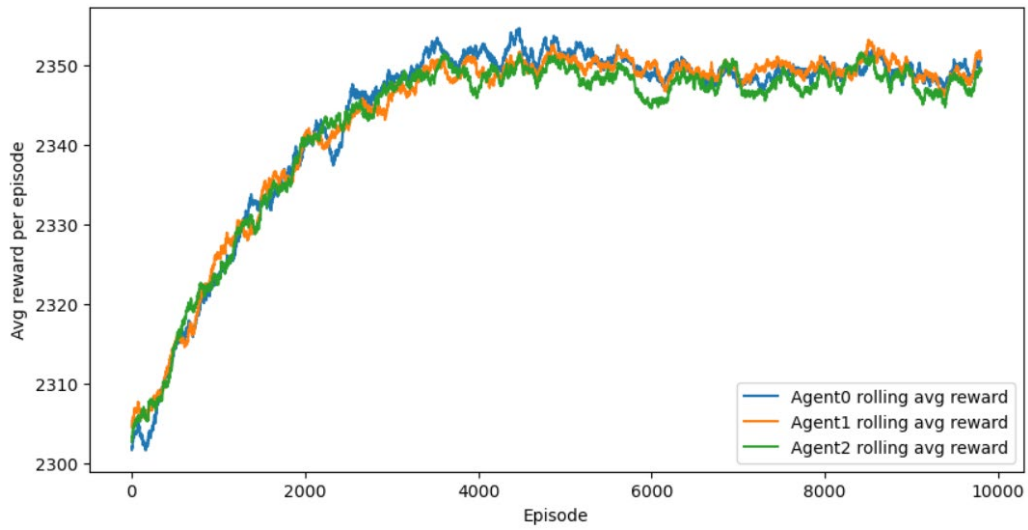


Overall, the results show that both algorithms are capable of learning and progressively improving their performance. All curves exhibit an upward trend, indicating that the agents increase their average reward as the episodes progress. Both Q-learning and DQN display two distinct behaviors: a higher trajectory reaching rewards close to 470, and a lower trajectory converging to values around 380. This suggests the possible existence of multiple local-optimal policies, as well as the influence of initial conditions and the exploration process.

In comparative terms, in a duopoly, Q-learning and DQN show very similar performance. The curves associated with both methods almost overlap throughout the training process, indicating that, for this particular environment, using deep neural networks does not provide a significant advantage over the tabular approach. In fact, Q-learning shows a slight advantage in the early training phase, demonstrating a faster increase in some cases.

The introduction of a third agent makes the learning environment more competitive. The Figure 4 shows the learning curves of three agents trained over 10 000 epsiodes using Q-learning. At the beginning of training, the exploration rate is very high (epsilon = 0.9995). In this phase, agents act mostly at random, which explains the relatively low and unstable initial rewards.

Figure 4 Training Q-learning progress - discrete oligopoly

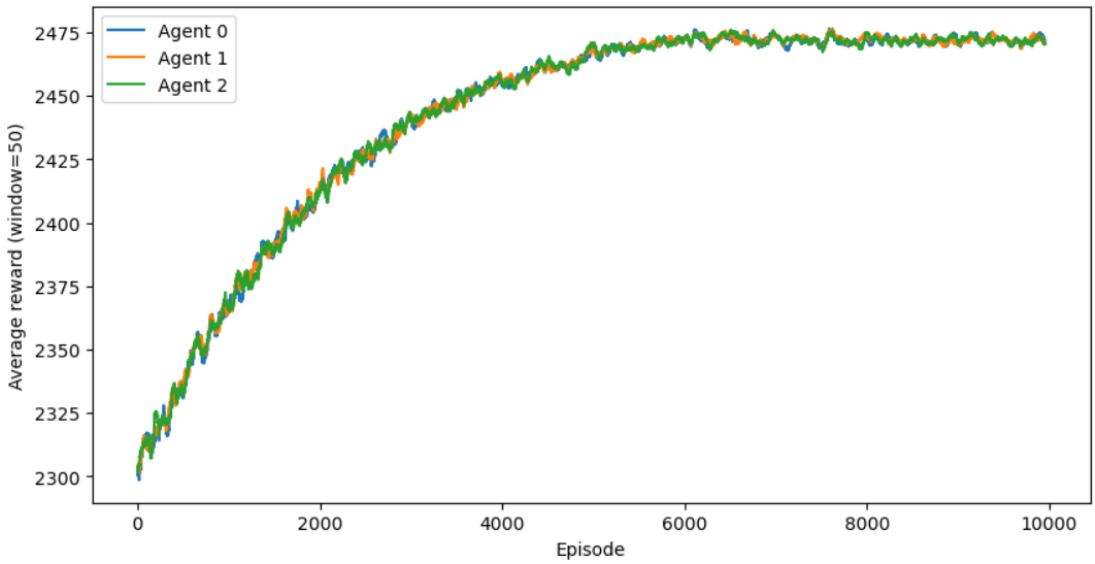


As training progresses, the exploration rate gradually decays, allowing the agents to rely on the knowledge accumulated in their Q-tables. Around episode 2 000, the average rewards of all three agents converge to similar values (approximately 2 336). This suggests that they are stabilizing their policies while still exploring.
From episode 3 500 to 4 500, it can be seen that Agent 0 temporarily outperforms the others. This reflects a period where its learned policy provides a slight advantage. However, the other agents eventually adapt, and the three learning curves converge again toward the same equilibrium level.

Overall, the learning curves show gradual convergence and stabilization. The small differences between the agent's rewards suggest that the environment allows for fair competition, and each agent is capable of learning optimal actions despite the presence of other learners.
This is coherent with economic theory: when multiple agents interact repeatedly, their strategies often converge to a Nash equilibrium where no agent can improve their payoff by unilaterally changing their strategy.

Figure 5 shows the learning curves of three agents trained using a Deep Q-Network (DQN) algorithm. The vertical axis reports the average reward computed over a rolling window of 50 episodes, while the horizontal axis represents the progression of training episodes.

Figure 5 Training DQN - discrete oligopoly

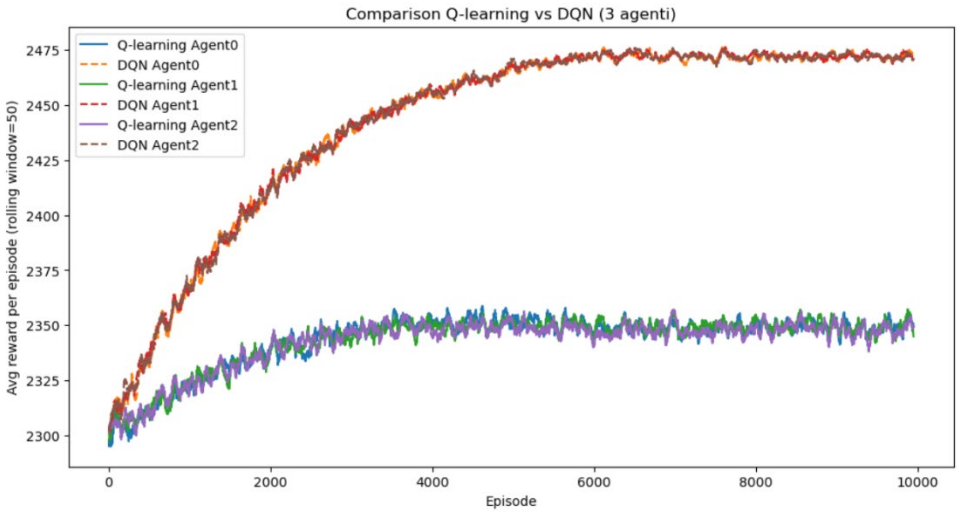


Initially, the agents obtain relatively low rewards due to high exploration. As the exploration rate decays, all three agents progressively improve their policies, leading to a steady rise in rewards. From around episode 6 000 onward, the curves remain flattened and aligned, indicating that the agents have converged to stable strategies with nearly identical payoffs.
Overall, the plot highlights the ability of the multi-agent DQN algorithm to reach a symmetric learning equilibrium. This underlies the economic principle that, in competitive repeated interactions, agents tend to converge to strategies that are mutually optimal rather than individually dominant.

Figure 6 compares the learning performance of the three agents trained with two different reinforcement learning algorithms: Q-learning and DQN.
A clear separation emerges between the two groups of curves, suggesting that the gap is due to the algorithms themselves. The DQN agents achieve significantly higher average

rewards, converging around 2 470 - 2 475, while the Q-learning agents stabilize at a lower level, around 2 330 - 2 340.

Figure 6 Comparison Q-learning vs DQN - discrete oligopoly



Beyond this performance gap, the learning dynamics of the two approaches also differ markedly. DQN agents exhibit fast and smooth improvement. In contrast, Q-learning agents progress more slowly and stabilize earlier, reflecting the limitations of tabular learning in more complex environments.

Within each algorithm, the three agent curves almost perfectly overlap. This symmetry indicates that all agents face identical learning conditions, no agent receives a structural advantage, and the environment allows them to converge toward similar equilibrium strategies.

From an economic perspective, this pattern reflects principles from game theory and repeated strategic interaction. According to the Nash equilibrium, agents continually adjust their behavior in response to both the environment and their competitors. A more flexible learning method like DQN enables them to approximate their best responses more effectively, leading to higher equilibrium payoffs.

Conclusion

Based on the results obtained, we observe that the agents tend to converge towards a Nash equilibrium. This behavior is consistent with the economic theory previously described: no agent can improve their payoff by unilaterally changing their pricing strategy once equilibrium is reached. This confirms that reinforcement learning agents can learn strategic interactions in imperfect competitive markets.

One limitation of the current model is that the demand function has been simplified to a linear form, which may not fully capture the complexities of real consumer behavior. Additionally, the action space is discrete. These simplifications may limit the generality of the results and the precision of the equilibrium analysis.

The model can be improved in several ways. Firstly, providing a continuous action space would allow agents to select prices in a more similar way to what happens in real markets. Second, extending the environment to include more agents would create a more general and realistic competitive scenario. Finally, introducing non-linear demand models would better capture consumer behavior and market dynamics.

Overall, the study demonstrates that reinforcement learning can successfully model strategic pricing behavior, but further improvements are necessary to fully capture realistic market dynamics.

References

Agiza, H. N., & Elsadany, A. A. (2003). Nonlinear dynamics in the Cournot duopoly game with heterogeneous players. *Physica A: Statistical Mechanics and Its Applications*, *320*, 512–524. https://doi.org/10.1016/S0378-4371(02)01648-5

Bischi, G. I., & Lamantia, F. (2022). Evolutionary oligopoly games with cooperative and aggressive behaviors. *Journal of Economic Interaction and Coordination*, *17*(1), 3–27. https://doi.org/10.1007/s11403-020-00298-y

Calvano, E., Calzolari, G., Denicolò, V., & Pastorello, S. (2020). Artificial Intelligence, Algorithmic Pricing, and Collusion. *American Economic Review*, *110*(10), 3267–3297. https://doi.org/10.1257/aer.20190623